

Episode 3: Catching Network Misconfigs Before They Hit Production

Video Title

"Network as Code: Four Layers of Validation That Save Your Change Window"

Target Length

18-22 minutes

Goal

Show the four validation layers in action. Demonstrate each layer catching a different class of error. Make it clear that validation is the safety net that makes everything else in the series trustworthy.

Pre-Recording Checklist

Before you hit record, make sure:

- Clean data model (all validations passing)
 - Prepare four "broken" versions of data files, one for each validation layer
 - Terminal ready with validation commands
 - Have a test run output ready as backup in case live runs take too long
-

SEGMENT 1: Recap and Setup (0:00 - 2:00)

What to show: Quick validation run showing everything passing.

```
uv run python validate.py
```

Talking points:

- "Last episode we built the data model. YAML files that describe what the network should look like. But YAML files are just text. Nothing stops you from putting garbage in them."
 - "Lab 2 adds four layers of validation that run before any config is generated, before any device is touched. If a mistake exists in the data model, it dies here."
 - "Let me start by showing you what a clean run looks like."
 - Show the passing output: "All checks passed. That is the green light. Now let me show you what happens when things go wrong."
-

SEGMENT 2: Layer 1 - Format Validation (2:00 - 4:30)

What to show: Break the YAML syntax and run validation.

Exact edit: In data/fabric.yaml, change line 9 from asn: 65000 to asn 65000 (remove the colon)

```
uv run python validate.py
```

Talking points:

- "Layer one is format validation. Can the YAML even be parsed? This catches typos, bad indentation, missing colons. Basic stuff, but it stops everything else if it fails."
- "This is the equivalent of 'does the code compile.' If your YAML is malformed, nothing downstream can read it."
- Show the error output: "Clear error message. The file, the line, what went wrong. Fix it and move on."
- "You might think this is too basic to mention. But I have seen outages caused by a copy-paste that broke YAML indentation. A format check would have caught it in seconds."

Revert: change asn 65000 back to asn: 65000

SEGMENT 3: Layer 2 - Syntax Validation (4:30 - 8:00)

What to show: Valid YAML, but wrong data types or out-of-range values.

Exact edit: In data/services/networks.yaml, change line 11 from vlan_id: 10 to vlan_id: 5000

```
uv run python validate.py
```

Talking points:

- "Layer two is syntax validation. The YAML parses fine, but do the values make sense? Is the ASN a valid number? Is the IP address actually an IP address? Is the VLAN ID in the valid range?"
- "This is where Pydantic does the heavy lifting. Every field in the data model has a type and constraints defined in the schema. Pydantic checks every single value against those constraints."
- Show the error: "SYN-03. The VLAN ID 5000 is outside the valid range of 1 to 4094. The validator tells you exactly which file, which field, and what the constraint is."
- "A YAML linter cannot catch this. YAML does not know what a VLAN ID is. But Pydantic does, because we told it."

Revert: change vlan_id: 5000 back to vlan_id: 10

SEGMENT 4: Layer 3 - Semantic Validation (8:00 - 13:00)

What to show: Valid YAML, valid types, but broken logic.

Demo 1: Missing Reference

Exact edit: In data/services/networks.yaml, change line 14 from vrf: PRODUCTION to vrf: STAGING

```
uv run python validate.py
```

- "SEM-04. A network references VRF 'STAGING' but that VRF is not defined in vrfs.yaml. This is a cross-file consistency check. The format is fine. The types are correct. But the logic is broken."

- "Deploy this to a device and it either errors out or silently creates a broken state. The semantic validator catches it before it leaves your laptop."

Revert: change vrf: STAGING back to vrf: PRODUCTION

Demo 2: Overlapping Subnets

Exact edit: In data/services/networks.yaml, change line 19 (prod-app subnet) from subnet: 10.100.20.0/24 to subnet: 10.100.10.0/25 (overlaps with prod-web's 10.100.10.0/24)

```
uv run python validate.py
```

- "SEM-02. Overlapping subnets detected. 10.100.10.0/24 and 10.100.10.0/25 overlap. On paper, someone might not notice. The validator does, every time."

Revert: change subnet: 10.100.10.0/25 back to subnet: 10.100.20.0/24

Demo 3: Asymmetric Links

Exact edit: In data/underlay.yaml, delete lines 54-61 (the entire spine2-to-leaf1 link block):

```
- name: spine2-to-leaf1
  a_device: spine2
  a_interface: eth1
  a_ip: 10.0.1.8/31
  b_device: leaf1
  b_interface: eth2
  b_ip: 10.0.1.9/31
  link_type: point_to_point
```

```
uv run python validate.py
```

- "SEM-07. Every fabric link should be defined from both sides. Spine2 should connect to leaf1, but the link definition is missing. This is the kind of inconsistency that only shows up when you deploy and one side of the link comes up while the other does not."

Revert: undo the deletion (Ctrl+Z) to restore the spine2-to-leaf1 link block

Talking points between demos:

- "These are the checks that matter most. Format and syntax validation are table stakes. Semantic validation is where you encode the design rules of your fabric."
- "Every one of these rules has a stable ID. SEM-01 through SEM-whatever. That means your CI/CD pipeline can track which rules are passing over time. You can add new rules without breaking existing checks."

SEGMENT 5: Layer 4 - Compliance Validation (13:00 - 16:00)

What to show: Run the compliance checks. If possible, trigger a compliance failure.

```
uv run python validate.py
```

Talking points:

- "Layer four is compliance. This is where organizational policy becomes code."
- "Semantic validation asks 'is this logically consistent?' Compliance validation asks 'does this meet our standards?'"

- "Examples: naming conventions. Every device name must follow the pattern. OSPF authentication is required on all fabric links. All management interfaces must be in the management VRF. No BGP peers without a password configured."
 - "These rules are different at every organization. The point is not the specific rules. The point is that you write them once as Python functions, and they run on every change, automatically. Tribal knowledge becomes enforceable policy."
 - "A new engineer joins the team and submits a change. They do not need to know every unwritten rule about how your network is designed. The compliance validator tells them."
-

SEGMENT 6: The Full Picture (16:00 - 18:00)

What to show: Run the full validation suite one more time with clean data.

```
uv run python validate.py
```

Talking points:

- "Four layers. Format, syntax, semantic, compliance. Each one catches a different class of error. Together, they form a wall between your data model and the network."
 - "In the CI/CD pipeline we build in Lab 4, these checks run automatically on every push. A failing validation blocks the merge. The network never sees a misconfiguration that the validator can catch."
 - "The Cisco NaC paper talks about shift-left testing. Moving validation as early as possible in the workflow. This is as far left as you can go. You catch the error before you even generate a config, let alone deploy it."
-

SEGMENT 7: pytest Integration (18:00 - 19:30)

What to show: Quick flash of running validations as pytest tests.

```
uv run pytest tests/test_format.py tests/test_syntax.py tests/test_semantic.py tests/test_compliance.py -v --tb=short
```

Talking points:

- "One more thing. All of these validations also run as pytest test cases. That gives you proper test reporting, HTML output, and integration with any CI/CD system that understands test results."
 - "The validate.py script is the quick check you run locally. The pytest suite is what runs in the pipeline with full reporting."
 - Show the test output briefly: "39 rules, all passing. Each with a stable test ID that maps to the validator rule."
-

SEGMENT 8: The Close (19:30 - 21:00)

What to show: Series overview or back to camera.

Talking points:

- "That is Lab 2. Four layers of validation that catch errors at every level: format, syntax, semantics, and compliance."

- "If you want to build this validation framework yourself, the full lab guide walks through writing every validator, every rule, and the pytest integration. Available at [your link] as part of the \$49 lab package."
 - "Next episode, we generate device configurations from the data model. Three tooling paths: Python with Jinja2, Ansible, and Terraform. I will show you the route reflector auto-peering demo that Cisco uses to sell the NaC concept."
 - "Subscribe, and I will see you in the lab."
-

Commands Reference (Quick Copy)

```
# Full validation
uv run python validate.py

# pytest validation suite
uv run pytest tests/test_format.py tests/test_syntax.py tests/test_semantic.py
tests/test_compliance.py -v --tb=short

# Individual layer
uv run pytest tests/test_format.py -v
uv run pytest tests/test_syntax.py -v
uv run pytest tests/test_semantic.py -v
uv run pytest tests/test_compliance.py -v
```

Do NOT Show On Camera

- The validator source code (paid content)
- The Pydantic schema definitions (paid content)
- How to write custom validation rules (paid content)
- The step-by-step build process (paid content)

DO Show On Camera

- Validation output (pass/fail messages, error details)
- Quick edits to data files to trigger failures (but not the full file contents)
- pytest test output and counts
- The four-layer concept and why each exists
- Error IDs and their stability across runs